How Many Steps to Get Numbers?

Three Paths — Pick the One That Fits Your Workflow

Option 1: Zero Steps

Every example ships with a pre-generated `METRICS_TABLE.md`

```
examples/yolov12/METRICS_TABLE.md
```

- CPU, GPU, NPU throughput & latency
- Cross-device comparison table
- Auto-generated from benchmark runs
- **Plan:** expose as a web dashboard

> **Just open the file — numbers are already there**

Option 2: Docker (One Command)

Pre-installed ROCm + RyzenAI environment

```
# Pull & run the container
docker run physical-ai-sdk

# Inside the container:
make benchmark-all-devices metric
```

- No local installs needed
- Reproducible environment
- Generates `METRICS_TABLE.md` automatically

> **One command to fresh numbers**

Option 3: Manual Setup

Full local installation — 4 steps:

```
# 1. Install ROCm
./install_rocm_stack.sh

# 2. Install RyzenAI
./install_ryzen_ai_source_stack.sh

# 3. Navigate to example
cd examples/yolov12

# 4. Run benchmarks
make benchmark-all-devices metric
```

Output: `METRICS_TABLE.md`

> **Most control, most setup**

Easy to Use

Uniform Interface Across Every Example

Make-Based Uniformity

Same commands work in **every** example directory:

```
make benchmark-all-devices metric   # all devices
make benchmark-gpu                  # GPU only
make benchmark-cpu                  # CPU only
make benchmark-npu                  # NPU only
make eval                           # evaluation pipeline
```

- Learn once, use everywhere
- No per-model setup surprises
- Consistent output format (`METRICS_TABLE.md`)

Every Example Includes:

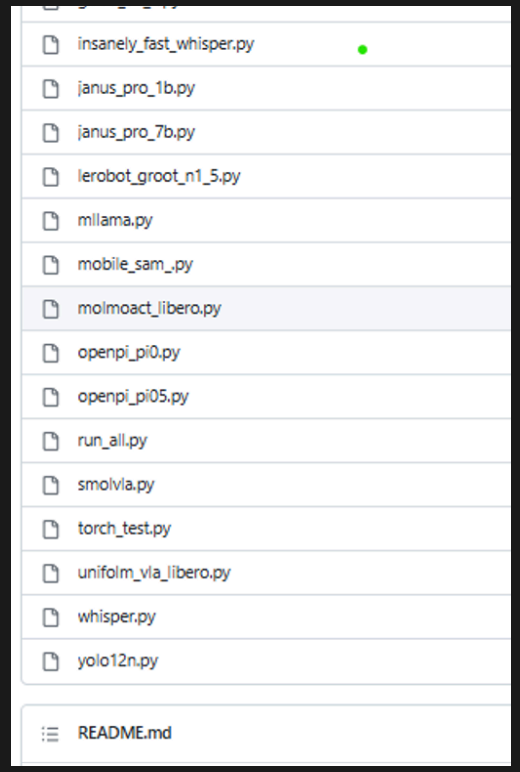
Feature	Description
CPU / GPU / NPU flows	Benchmark each compute device for throughput & latency
Eval pipeline	Sample inputs → sample outputs to visually verify correctness
Guiding documentation	Per-example tutorial to walk you from setup to results

> **Principle:** If you can run one example, you can run them all — same commands, same structure, same outputs.

Contributions — Research Desk → Vertical Software

1. PAVS transforms raw researcher code into production-ready vertical software

What we receive: Research Repo



Flat list of scripts — no structure, no device flows, no Make targets

What we receive: Research Code

```
# rocm-scripts/test/pytorch/yolo12n.py
#........
# .......
# .......
#........
# .......
# .......
recalled = gt_labels & detected_classes
recall = len(recalled) / len(gt_labels)
sanity_pass = recall >= RECALL_THRESHOLD
assert sanity_pass, (
  f"recall {recall:.0%} < {RECALL_THRESHOLD:.0%}")

return lats, {
  "architecture": "CNN",
  "task": "object_detection",
  "num_detections": num_detections,
  "recall": round(recall, 4),
}
if __name__ == "__main__":
  sys.exit(run(run_benchmark))
```

Single script, no structure, no device flows

What we deliver: Vertical Software

```
examples/yolov12/
├── Makefile
├── METRICS_TABLE.md
├── README.md
├── app/
│   ├── routes/
│   ├── services/
│   ├── main.py & config.py
├── scripts/
│   ├── benchmark.py
│   ├── export_onnx.py
│   ├── profile_model.py
│   ├── run_val.py
│   └── update_metrics_table.py
├── tests/
│   ├── test_api.py
│   ├── test_docker.py
│   ├── test_migraphx.py
│   └── test_yolo_workflow.py
├── weights/
├── datasets/
├── requirements.txt
├── requirements_cpu.txt
└── requirements_npu.txt
```

App, scripts, tests, per-device requirements — production-ready

Physical AI & Vertical Software: Researcher's Desk → Vertical Software — same commands, every device, production-ready.

AMD
together we advance_

Contributions(contd.) — Building the AMD AI Ecosystem

2. Developer-Friendly Inference Tools

Profilers for cloud & edge — with tutorials for ease of adoption:

Tool	Purpose
Lemonade	End-to-end model profiling
NPU AI Analyzer	NPU workload analysis
ROCProfiler	GPU kernel-level profiling

Bridging AMD Tools to Opensource Ecosystems

3. LLM Infrastructure on AMD

Making LLMs easy on AMD hardware:

- **vLLM** — high-throughput serving
- **Llama.cpp** — efficient edge inference
- Bridging **open-source ecosystem** ↔️ **AMD HW ecosystem**

4. Vertical Application Platform

A platform that **evolves with the client** — solving, fixing, and growing together in targeted domains:

Domain	Focus
Healthcare	Medical imaging, diagnostics
Automotive	Perception, safety systems
Industrial	Inspection, anomaly detection

> **Building Real ROI around AI** — not demos, but Physical AI verticals that deliver measurable business value.

Pain Points

Current Challenges on Edge Devices

ROCm Installation Stability

- ROCm install on edge devices is **not yet stable**
- Driver/firmware mismatches on some hardware revisions
- Workaround: pinned ROCm versions in install scripts

Reboot During Installation

- ROCm installation requires a **system reboot** mid-process
- Breaks unattended / CI-driven installs
- Users unfamiliar with the flow find this unexpected

Sudo / Root Access Required

- Both ROCm and RyzenAI installers need **sudo privileges**
- Containers mitigate this

> **Docker path sidesteps all three issues** — pre-built image with ROCm + RyzenAI already configured, no reboot, no sudo on host.

SW Stack (For Reference)